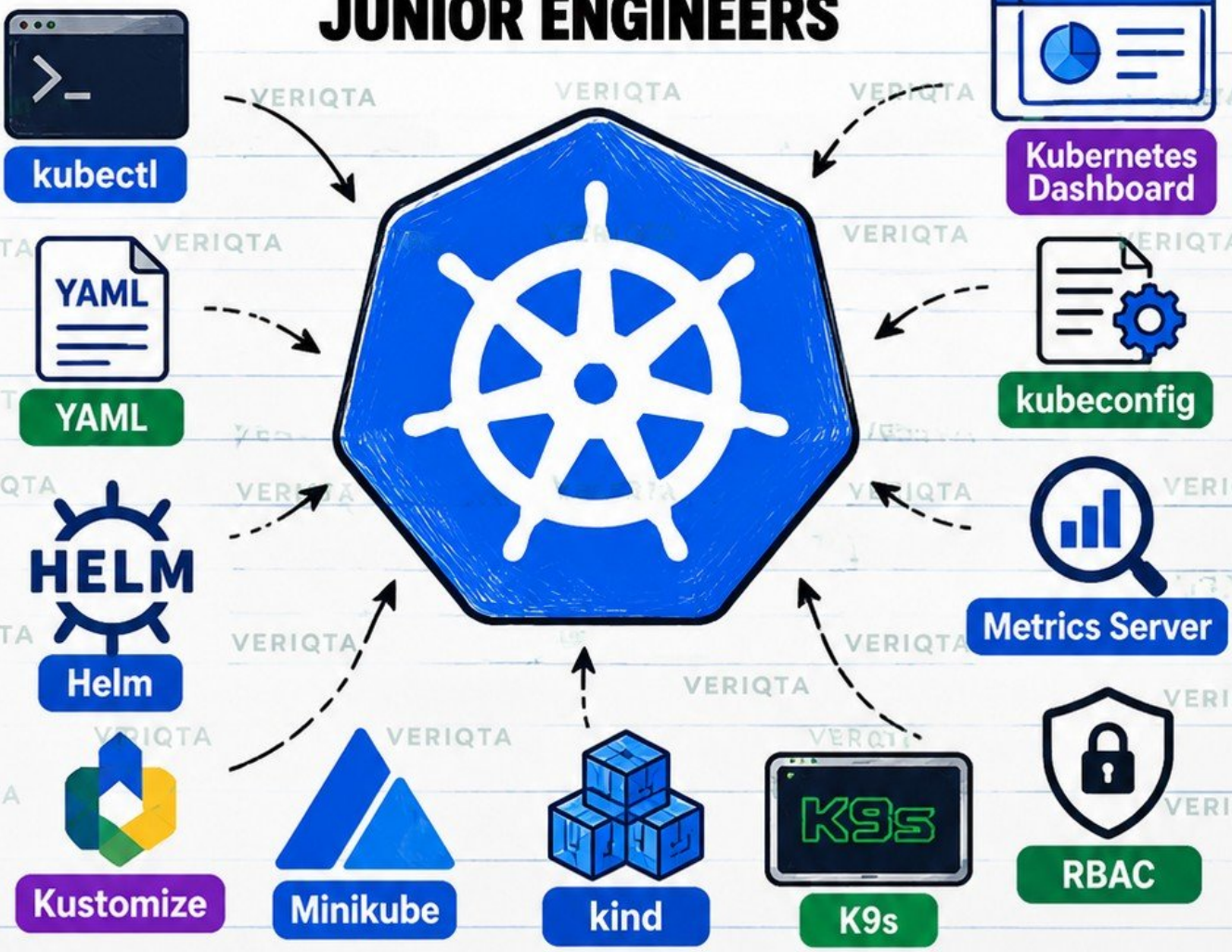
VERIQTA

# KUBERNETES TOOLKIT

## ESSENTIAL TOOLS FOR JUNIOR ENGINEERS




@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

# KUBERNETES TOOLKIT OVERVIEW

## Essential Tools for Junior Engineers

**A Kubernetes engineer uses a set of tools to create, manage, inspect, and troubleshoot applications on a Kubernetes cluster. Each tool has a specific purpose and together they make working with Kubernetes easier and more efficient.**

## Why Do We Need These Tools?

- Kubernetes itself is a powerful platform, but it can be complex.
- These tools help you interact with the cluster, deploy applications, manage configurations, monitor resources, and troubleshoot problems.
- Engineers use different tools depending on the task and environment (development, testing, or production).

## Key Tools in the Kubernetes Toolkit

### kubectl
- Main command-line tool to interact with Kubernetes.
- Create, inspect, update and manage resources.
- Used in every environment.

### YAML
- Define Kubernetes resources.
- Write application configurations.
- Used with kubectl, Helm and Kustomize.

### Helm
- Package manager for Kubernetes.
- Install and manage applications (charts).
- Useful for complex applications.

### Minikube
- Runs a local Kubernetes cluster on your computer.
- Great for learning and testing.
- Simple and lightweight.

### Kustomize
- Manage environment-specific configurations.
- Customize YAML files without duplicating them.
- Used for development and production setups.



**Kubernetes Toolkit**

### kind
- Runs Kubernetes clusters in Docker containers.
- Useful for testing and CI.
- Close to real Kubernetes environment.

### Kubernetes Dashboard
- Web UI to view and manage resources.
- Useful for visual inspection.
- Good for beginners and troubleshooting.

### K9s
- Terminal-based UI.
- Navigate, inspect and manage resources quickly.
- Useful for day-to-day work.

### Metrics Server
- Collects CPU and memory usage for pods and nodes.
- Used with kubectl top.
- Helps with monitoring and troubleshooting.

### kubeconfig
- Configuration file to connect to clusters.
- Contains clusters, users and contexts.
- Select the correct cluster and namespace.

## Junior Engineer Takeaway

- Each tool solves a specific problem.
- You will rarely use only one tool. Most real tasks involve multiple tools.
- Start with kubectl, YAML and a local cluster (Minikube or kind), then explore Helm, Kustomize and the other tools.

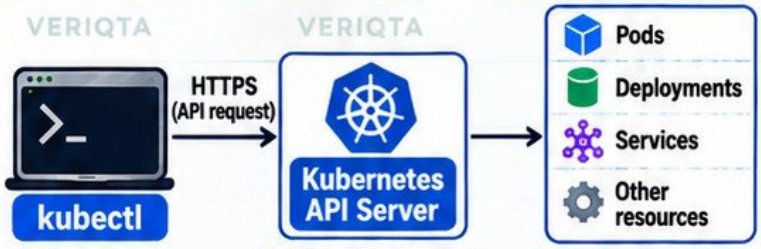
VERIQTA

# kubectl: The Kubernetes Command-Line Tool

## What is kubectl?

- kubectl is the command-line tool used to interact with Kubernetes clusters.
- It sends commands to the Kubernetes API server to create, inspect, update, and manage resources.
- Engineers use kubectl in development, testing, and production environments.

## How kubectl Communicates




You run a command → Validates the request and performs the action → Kubernetes cluster state is updated or returned

## kubectl Command Structure

**kubectl** [command] [resource] [name] [flags]

- command: action to perform (get, create, apply, delete, etc.)
- resource: Kubernetes resource type (pods, deployments, services, etc.)
- name: name of the resource (optional)
- flags: additional options (e.g., -n for namespace, -o for output format)

## Example

**kubectl get pods -n dev -o wide**

- kubectl: the tool
- get: command to view resources
- pods: resource type
- -n dev: namespace (dev)
- -o wide: output format (detailed information)

## Essential kubectl Commands

| Command | What It Does | Example | When to Use It |
|---|---|---|---|
| kubectl version | Shows client and server version | kubectl version | Check cluster and client versions |
| kubectl get | List resources | kubectl get pods | View existing resources |
| kubectl describe | Show detailed information | kubectl describe pod my-pod | Investigate resource details |
| kubectl create | Create a resource | kubectl create -f pod.yaml | Create resources from YAML |
| kubectl apply | Create or update a resource | kubectl apply -f deployment.yaml | Deploy or update resources |
| kubectl delete | Delete a resource | kubectl delete pod my-pod | Remove a resource |
| kubectl logs | View container logs | kubectl logs my-pod | Troubleshoot application issues |
| kubectl exec | Run a command in a container | kubectl exec -it my-pod -- /bin/bash | Access a running container |
| kubectl get all | View common resources | kubectl get all -n dev | Quickly see resources in a namespace |
| kubectl config | Manage kubeconfig settings | kubectl config get-contexts | Check or switch cluster contexts |

## Common Beginner Mistakes

- Using the wrong namespace (forgetting -n).
- Typing the wrong resource name (e.g., pod vs pods).
- Expecting immediate changes without checking the output.
- Running commands without understanding the impact.
- Using kubectl delete on important resources by mistake.

## Junior Engineer Tip

- Use kubectl get to confirm the current state before making changes.
- Use -n <namespace> to avoid working in the wrong namespace.
- Use descriptive names and check the output for errors.

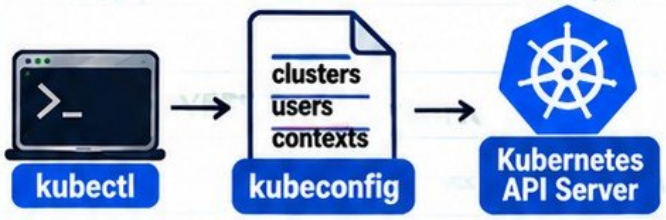
VERIQTA

# kubeconfig: Connecting to Kubernetes Clusters

## What is kubeconfig?

- kubeconfig is a configuration file that stores information about Kubernetes clusters, users, and contexts.
- It tells kubectl how to connect to a cluster and which user credentials to use.
- You can have multiple clusters in one kubeconfig file (e.g., dev, staging, production).
- Engineers use it to safely switch between environments.

## How It Works




## Key Sections in kubeconfig

| Section | What It Does |
|---|---|
| clusters | Contains cluster details (name, API server URL, certificate information). |
| users | Contains authentication information (e.g., tokens, certificates, or exec plugins). |
| contexts | Combines a cluster and a user so you can switch between environments. |
| current-context | Defines which context kubectl uses by default. |

## Simple kubeconfig Structure

```
apiVersion: v1
kind: Config
clusters:
- name: dev-cluster
  cluster:
    server: https://dev.example.com
    certificate-authority: /path/to/ca.crt
users:
- name: dev-user
  user:
    token: YOUR_TOKEN_HERE
contexts:
- name: dev-context
  context:
    cluster: dev-cluster
    user: dev-user
current-context: dev-context
```

Cluster information

User credentials

Context combines cluster + user

Default context

## Useful kubectl Commands

| Command | What It Does | Example |
|---|---|---|
| kubectl config view | Show your current config | kubectl config view |
| kubectl config get-clusters | List all clusters | kubectl config get-clusters |
| kubectl config get-users | List all users | kubectl config get-users |
| kubectl config get-contexts | List all contexts | kubectl config get-contexts |
| kubectl config current-context | Show current context | kubectl config current-context |
| kubectl config use-context | Switch context | kubectl config use-context dev-context |
| kubectl config set-context | Create or update a context | kubectl config set-context my-context --cluster=prod --user=me |

## Switching Between Clusters

1. List available contexts:
   `kubectl config get-contexts`
2. Check the current context:
   `kubectl config current-context`
3. Switch to a different context:
   `kubectl config use-context staging`
4. Verify you are connected to the right cluster:
   `kubectl get nodes`

## Common Beginner Mistakes

- Working on the wrong cluster or environment.
- Using outdated or expired credentials.
- Editing kubeconfig manually without understanding it.
- Sharing kubeconfig files (risk of exposing credentials).
- Forgetting to verify the current context before making changes.

## Junior Engineer Tip

- Always check the current context before running any command that changes resources.
- Use clear and meaningful names for your contexts (e.g., dev, staging, prod).
- Keep separate kubeconfig files for different environments if needed.
- Protect your kubeconfig file. It can give full access to your clusters.

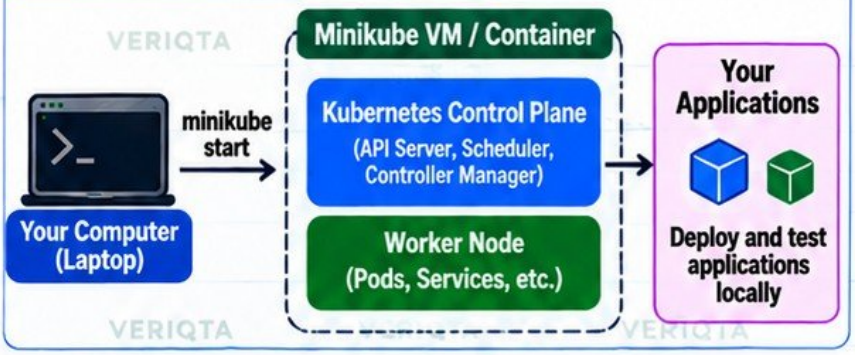
VERIQTA

# Minikube: Your Local Kubernetes Lab

## What is Minikube?

- Minikube runs a single-node Kubernetes cluster on your local machine.
- It creates a complete Kubernetes environment for learning, testing, and development.
- It is easy to set up and lightweight.
- You can deploy real applications and practice all the essential Kubernetes tools and commands.

## How Minikube Works




## Install Minikube

1. Download Minikube (Linux example):
   ```
   curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
   ```
2. Make it executable:
   ```
   sudo chmod +x minikube-linux-amd64
   ```
3. Move to a directory in your PATH:
   ```
   sudo mv minikube-linux-amd64 /usr/local/bin/minikube
   ```
4. Verify installation:
   ```
   minikube version
   ```

## Essential Minikube Commands

| Command | What It Does | Example |
|---|---|---|
| minikube start | Starts a local cluster | minikube start |
| minikube status | Shows cluster status | minikube status |
| minikube stop | Stops the cluster | minikube stop |
| minikube delete | Deletes the cluster | minikube delete |
| minikube dashboard | Opens Kubernetes Dashboard | minikube dashboard |
| minikube service | Accesses a service in your browser | minikube service my-service |
| minikube ip | Shows the node IP | minikube ip |
| minikube addons list | Lists available addons | minikube addons list |
| minikube addons enable | Enables an addon | minikube addons enable metrics-server |

## Check Your Cluster

After starting Minikube, verify it is working:

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info
kubectl get namespaces
```

## Access the Dashboard

1. Start the dashboard:
   ```
   minikube dashboard
   ```
2. A browser window will open with the Kubernetes UI.
3. You can view and manage your local cluster (resources, pods, deployments, services, etc.).




## Useful Addons

- metrics-server – enable resource metrics
  `minikube addons enable metrics-server`
- ingress – practice Ingress resources
  `minikube addons enable ingress`
- dashboard – graphical UI
  `minikube addons enable dashboard`

## Common Beginner Mistakes

- Not starting Minikube before running kubectl commands.
- Forgetting to enable required addons (e.g., metrics-server).
- Using Minikube for production (it is only for learning and testing).
- Assuming Minikube and a real production cluster are the same.
- Not checking the cluster status when commands fail.

## Junior Engineer Tip

- Use Minikube to practice as much as possible.
- Try deploying real applications.
- Experiment with different addons.
- When you are comfortable, move to kind or a cloud-based cluster for more advanced scenarios.

VERIQTA

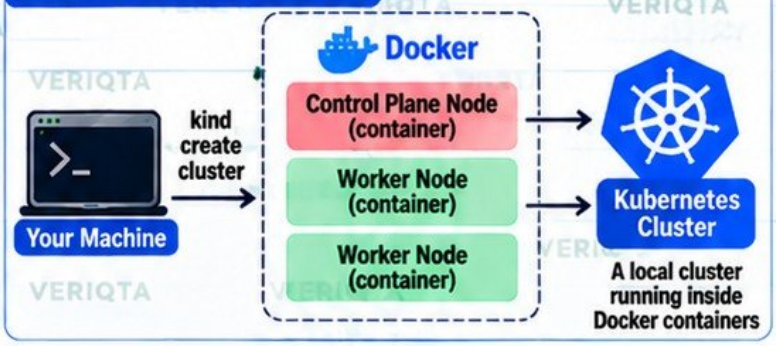
# kind: Kubernetes in Docker

**Run local Kubernetes clusters using Docker containers. kind is simple, lightweight and perfect for testing, learning and CI environments.**

## What is kind?

- kind (Kubernetes IN Docker) runs Kubernetes clusters using Docker containers.
- Each node (control plane and workers) is a Docker container.
- It gives you a real Kubernetes cluster that works like a production cluster.
- It is lightweight, easy to set up and perfect for learning, testing and CI.

## How kind Works




## Install kind

1. Download kind (Linux example):
   ```
   curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64
   ```
2. Make it executable:
   ```
   chmod +x ./kind
   ```
3. Move to a directory in your PATH:
   ```
   sudo mv ./kind /usr/local/bin/kind
   ```
4. Verify installation:
   ```
   kind version
   ```

## Create a Cluster

1. Create a simple cluster:
   ```
   kind create cluster
   ```
2. Check the cluster:
   ```
   kubectl cluster-info
   ```
3. List nodes:
   ```
   kubectl get nodes
   ```
4. List all resources:
   ```
   kubectl get all -A
   ```

**Example Output**

```
$ kind create cluster
Creating cluster "kind"
✓ Ensuring node image
✓ Pulling nodes
✓ Writing configuration
✓ Starting control-plane
✓ Installing CNI
✓ Done! kubectl is configured to use "kind" cluster.
```

## Custom Cluster (Multiple Nodes)

Create a cluster with 1 control plane and 2 worker nodes using a config file.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

Create the cluster:

```
kind create cluster --config kind-config.yaml
```

## Useful Commands

| Command | What It Does | Example |
|---|---|---|
| kind create cluster | Creates a cluster | kind create cluster |
| kind get clusters | List all local clusters | kind get clusters |
| kind delete cluster | Delete a cluster | kind delete cluster |
| kind export kubeconfig | Export kubeconfig | kind export kubeconfig --name mycluster |
| docker ps | View kind nodes (containers) | docker ps |
| kubectl get nodes | List cluster nodes | kubectl get nodes |
| kubectl get all -A | List all resources | kubectl get all -A |

## Why Engineers Use kind

- Test Kubernetes manifests and applications.
- Practice Kubernetes without a cloud account.
- Use it in CI pipelines for automated testing.
- Quickly create and destroy clusters.
- Simulate multi-node environments.

## Common Beginner Mistakes

- Forgetting Docker must be installed and running.
- Assuming kind is for production (it is for local use).
- Not checking cluster status after creation.
- Using the wrong kubeconfig context.
- Naming conflicts with existing clusters.

## Junior Engineer Tip

- Use kind to test your YAML files before deploying to real clusters.
- Create and delete clusters often to keep your environment clean.
- Practice with multi-node clusters to understand real-world setups.

VERIQTA

# YAML: Defining Kubernetes Resources

**Kubernetes uses YAML files, called manifests, to define and manage resources such as Pods, Deployments, Services and more.**

## What is a Kubernetes Manifest?

- A manifest is a YAML file that describes the desired state of a Kubernetes resource.
- You tell Kubernetes what you want, and it creates or updates the resource.
- Manifests are used with kubectl apply to deploy applications and configurations.
- YAML is human-readable and easy to version control with Git.

## How It Works




## Key Sections in a Kubernetes Manifest

| Section | What It Does |
|---|---|
| apiVersion | Specifies the Kubernetes API version for the resource type. |
| kind | Type of resource (Pod, Deployment, Service, etc.). |
| metadata | Basic information like name, namespace and labels. |
| spec | Desired state of the resource (configuration details). |

## Example: Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

API version

Resource type

Metadata (name, namespace)

Desired state (configuration)

## Common Resource Kinds

**Pod**
Runs one or more containers

**Deployment**
Manages replica Pods and updates

**Service**
Exposes applications

**ConfigMap**
Stores non-sensitive configuration

**Secret**
Stores sensitive data (e.g. passwords)

**Namespace**
Logical separation of resources

**PersistentVolume**
Provides storage in the cluster

**Ingress**
Manages external access to services

## Apply a Manifest

```
# Create or update the resource
kubectl apply -f deployment.yaml

# Check the resource
kubectl get deployments

# View full details
kubectl describe deployment my-app
```

## Common Beginner Mistakes

- Using the wrong apiVersion for the resource.
- Misspelling the kind or field names.
- Incorrect indentation in YAML.
- Missing required fields in spec.
- Using tabs instead of spaces.
- Forgetting to set a namespace.

## Junior Engineer Tip

- Start with simple manifests and gradually add more fields.
- Use `kubectl explain <resource>` to see available fields.
- Validate your YAML before applying:
  `kubectl apply --dry-run=client -f file.yaml`
- Keep your manifests in Git to track changes.

VERIQTA

# kubectl apply: Deploying Applications

**Use kubectl apply to create and update Kubernetes resources using declarative manifests, and verify the results.**

## What is kubectl apply?

- kubectl apply creates or updates Kubernetes resources from a YAML file (declarative approach).
- If the resource does not exist, it is created.
- If it already exists, it is updated with the new configuration.
- It compares the desired state in the YAML file with the current state in the cluster.
- It is the recommended way to manage Kubernetes resources.

## How kubectl apply Works




## Basic Command Syntax

```
kubectl apply -f <filename>
```

- `-f` : specifies the YAML file to use
- You can apply a single file or a directory
- Works for all resource types (Deployments, Services, etc.)
- Safe to run multiple times (idempotent)

## Example: Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

API version (resource type version)

Resource type

Resource name and labels

Desired state (replicas and selector)

Pod template (configuration)

## Common kubectl apply Commands

| Command | What It Does | Example |
|---|---|---|
| kubectl apply -f file.yaml | Create or update a resource from a file | kubectl apply -f deployment.yaml |
| kubectl apply -f . | Apply all YAML files in current directory | kubectl apply -f . |
| kubectl apply -k <dir> | Apply a Kustomize directory | kubectl apply -k ./overlays/dev |
| kubectl apply --dry-run=client -f file.yaml | Check for errors without creating resources | kubectl apply --dry-run =client -f deployment.yaml |
| kubectl apply -f file.yaml --validaten=false | Apply without schema validation (not recommended) | kubectl apply -f file.yaml --validate=false |

## Deploy and Verify (Step by Step)

1. Create the YAML file (e.g., deployment.yaml).
2. Apply the manifest:
   `kubectl apply -f deployment.yaml`
3. Check the resource:
   `kubectl get deployments`
4. Verify Pods are running:
   `kubectl get pods`
5. View more details if needed:
   `kubectl describe deployment my-app`

## Update an Application

Edit the YAML file (e.g., change replicas from 3 to 5) and apply it again.

```
# Change replicas from 3 to 5
spec:
   replicas: 5

# Apply the changes
kubectl apply -f deployment,yaml

# Verify the update
kubectl get pods
```

## Common Beginner Mistakes

- Using the wrong file path.
- YAML syntax errors.
- Applying to the wrong namespace.
- Forgetting required fields.
- Expecting immediate changes (the update may take a few seconds).
- Using kubectl create instead of apply for updates.

## Junior Engineer Tip

- Use kubectl apply for all declarative Kubernetes resources.
- Keep your YAML files in version control (Git).
- Run `kubectl apply --dry-run=client` first to catch errors early.

VERIQTA

# Helm: The Kubernetes Package Manager

**Helm helps you install, manage and upgrade complex Kubernetes applications using pre-configured packages called charts.**

## What is Helm?

- Helm is the official package manager for Kubernetes.
- It uses charts to package Kubernetes manifests, making it easy to deploy and manage applications.
- A chart is a collection of templates, configuration files and metadata.
- Helm simplifies installation, upgrades, rollbacks and configuration management.

## How Helm Works




**Helm Chart** — A packaged set of Kubernetes templates

**Values File** — Provides configuration (e.g. image, replicas)

**Helm** — Combines chart and values, creates resources

**Kubernetes Cluster** — Application deployed and managed

## Key Concepts

| Concept | What It Means |
|---|---|
| Chart | A package containing Kubernetes manifests, templates and metadata. |
| Release | A running instance of a chart in a cluster. |
| Values File | A YAML file used to configure the chart (e.g. replica count, image, resources). |
| Repository | A location (e.g. stable, bitnami) where charts are stored. |
| Template | A YAML file with variables that are replaced with values during installation. |
| Helm CLI | The command-line tool used to manage charts and releases. |

## Helm Chart Structure

```
mychart/
├── Chart.yaml      # Chart metadata
├── values.yaml     # Default configuration
├── templates/      # Kubernetes manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
├── charts/         # (Optional) subcharts
└── README.md       # Documentation
```

## Example values.yaml

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: 1.25
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 250m
    memory: 256Mi
```

## Common Helm Commands

| Command | What It Does | Example |
|---|---|---|
| helm repo add | Add a chart repository | helm repo add bitnami https://charts.bitnami.com/bitnami |
| helm repo update | Update local repo index | helm repo update |
| helm search repo | Search for charts | helm search repo nginx |
| helm install | Install a chart | helm install myapp bitnami/nginx |
| helm list | List installed releases | helm list |
| helm upgrade | Upgrade a release | helm upgrade myapp bitnami/nginx |
| helm rollback | Rollback to a previous version | helm rollback myapp 1 |
| helm uninstall | Remove a release | helm uninstall myapp |
| helm show values | View default values | helm show values bitnami/nginx |
| helm show chart | View chart details | helm show chart bitnami/nginx |

## Install a Real Application (Nginx)

1. Add a repository:
   `helm repo add bitnami https://charts.bitnami.com/bitnami`
2. Update the repository:
   `helm repo update`
3. Install Nginx:
   `helm install myapp bitnami/nginx`
4. Check the release:
   `helm list`
5. Verify the resources:
   `kubectl get all -l app.kubernetes.io/instance=myapp`

**Nginx** — Deployed using Helm chart

## Why Use Helm?

- Simplifies deployment of complex applications.
- Easy configuration using values files.
- Manage application lifecycles (install, upgrade, rollback).
- Access to a large ecosystem of pre-built charts (e.g. Bitnami, Prometheus, Grafana).
- Reusable and versioned deployments.

## Common Beginner Mistakes

- Using the wrong chart or repository.
- Not customizing values for your environment.
- Forgetting to run `helm repo update`.
- Not checking the release status after install.
- Assuming Helm replaces kubectl (it uses kubectl under the hood).
- Not cleaning up test releases.

## Junior Engineer Tip

- Always check the values file and customize it for your environment.
- Use `helm install --dry-run --debug` to see the resources before installing.
- Start with simple charts like nginx.
- Explore popular repos like `bitnami` and stable.
- Keep track of your releases using `helm list`.

VERIQTA

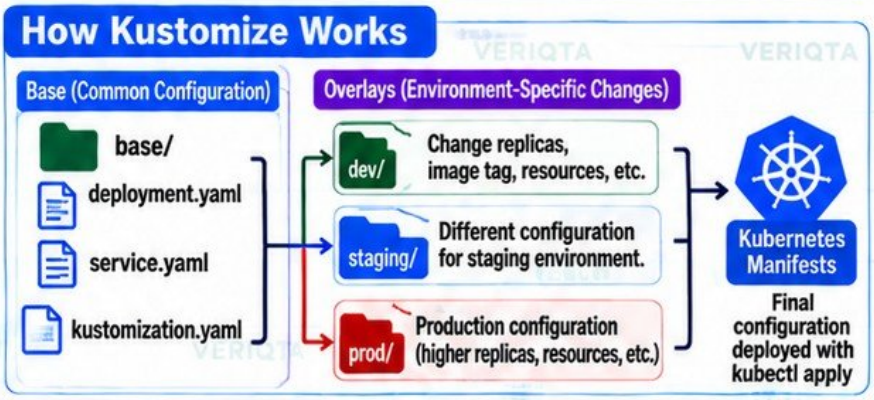
# Kustomize: Managing Kubernetes Configurations

**Kustomize helps you manage Kubernetes configurations using a base and environment-specific changes without duplicating manifests.**

## What is Kustomize?

- Kustomize is a Kubernetes-native tool for customizing YAML manifests.
- It allows you to manage a base configuration and apply environment-specific changes using overlays.
- You can modify resources without duplicating entire files.
- It works with kubectl and does not require a separate installation (included in kubectl).

## How Kustomize Works




## Project Structure Example

```
my-app/
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/
    │   ├── kustomization.yaml
    │   └── patch-replicas.yaml
    ├── staging/
    │   ├── kustomization.yaml
    │   └── patch-image.yaml
    └── prod/
        ├── kustomization.yaml
        └── patch-resources.yaml
```

## Base kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
labels:
  - pairs:
      app: my-app
```

## Overlay kustomization.yaml (dev)

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
patchesStrategicMerge:
  - patch-replicas.yaml
images:
  - name: my-app
    newTag: dev
```

## Base deployment.yaml (snippet)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: my-app:latest
```

## Overlay patch-replicas.yaml (dev)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
```

## Common Kustomize Commands

| Command | What It Does |
|---|---|
| kubectl kustomize <dir> | Builds the final YAML (manifest output) |
| kubectl apply -k <dir> | Apply the kustomized resources |
| kubectl diff -k <dir> | Show changes before applying |
| kustomize build <dir> | Generate manifest (using kustomize directly) |

## Finalize and Apply

```
# Preview the final manifest for dev
kubectl kustomize overlays/dev

# Apply to the cluster
kubectl apply -k overlays/dev

# Apply to staging
kubectl apply -k overlays/staging

# Apply to production
kubectl apply -k overlays/prod
```

## Why Use Kustomize?

- Manage multiple environments (dev, staging, prod).
- Avoid duplicating YAML files.
- Keep a single source of truth.
- Easy customization using overlays and patches.
- Works directly with kubectl (no extra tools).
- Ideal for GitOps and CI/CD workflows.

## Common Beginner Mistakes

- Duplicating YAML files instead of using overlays.
- Incorrect relative paths in kustomization.yaml.
- Forgetting to include the base in overlays.
- Using patches with wrong field names.
- Not running kubectl kustomize to verify output.
- Overcomplicating with too many overlays.

## Junior Engineer Tip

- Start with a simple base and one overlay (dev) first.
- Use `patchesStrategicMerge` to modify specific fields.
- Keep overlays minimal and organized.
- Always run `kubectl kustomize <dir>` to preview the final manifest.
- Use clear directory names (dev, staging, prod).

VERIQTA

# Kubernetes Dashboard and Headlamp

**Explore graphical tools for inspecting Kubernetes workloads, namespaces, resources, and cluster health.**

## What Are They?

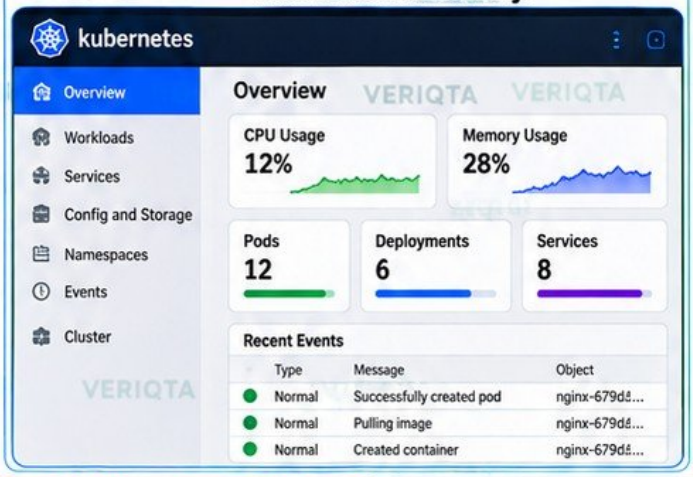
- Kubernetes Dashboard is the official web UI provided by the Kubernetes community.
- Headlamp is an open source, modern Kubernetes UI with more features and plugin support.
- Both tools help you visually inspect and manage your cluster resources.
- They are useful for learning, troubleshooting and day-to-day operations.

## When to Use Them?

- Explore and understand cluster resources visually.
- Check the status of Pods, Deployments, Services, etc.
- Troubleshoot issues quickly.
- Monitor cluster health and resource usage.
- Useful for beginners and experienced engineers.

## Kubernetes Dashboard

Kubernetes Dashboard is a graphical user interface (web UI) for managing Kubernetes clusters. It is officially maintained by the Kubernetes community.




## Headlamp

Headlamp is an open source, modern and extensible Kubernetes UI. It provides a clean interface with more features and plugin support.




## Install Kubernetes Dashboard

1. Apply the Dashboard manifest:
   ```
   kubectl apply -f https://raw.githuburesttort.com/kubernetes/dashboard/v2.7.0/aio/dashboard.yaml
   ```
2. Access the Dashboard:
   ```
   kubectl proxy
   ```
   Open in browser:
   ```
   http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard:proxy/
   ```
3. Login using a service account token (for learning environments).
4. Explore the UI.

## Install Headlamp

1. Download Headlamp (binaries available):
   ```
   https://headlamp.dev
   ```
2. Install on Linux:
   ```
   tar -xzf headlamp-linux-amd64.tar.gz
   sudo mv headlamp /usr/local/bin/
   ```
3. Run Headlamp:
   ```
   headlamp
   ```
4. Open in browser:
   ```
   http://localhost:4466
   ```
5. Connect to your cluster (using kubeconfig).

## Key Features Comparison

| Feature | Kubernetes Dashboard | Headlamp |
|---|---|---|
| Official project | ✓ | ✗ |
| Modern UI | ✗ | ✓ |
| Plugin support | ✗ | ✓ |
| Resource overview | ✓ | ✓ |
| View & manage workloads | ✓ | ✓ |
| Namespace management | ✓ | ✓ |
| Cluster health info | ✓ | ✓ |
| Extensibility | ✗ | ✓ |
| Best for | Basic usage | Advanced usage |

## Common Use Cases

- Inspect Pods, Deployments, Services, ConfigMaps and Secrets.
- Check resource usage (CPU, Memory).
- View logs and events.
- Debug failing workloads.
- Explore namespaces and RBAC.
- Quickly understand cluster status.

## Important Notes

- Use in secure environments.
- Enable authentication and RBAC.
- Do not expose the Dashboard or Headlamp publicly without proper protection.
- Use them for learning, development and internal cluster access.

VERIQTA

# kubectl get and describe: Inspecting Resources

**Learn how to examine Pods, Deployments, Services, resource status, and Kubernetes events using kubectl get and kubectl describe.**

## What Are These Commands?

- kubectl get shows a summary of Kubernetes resources.
- kubectl describe shows detailed information about a specific resource.
- They help you inspect the current state, configuration, and events.
- Essential for troubleshooting and understanding what is running in your cluster.

## Command Syntax

```
kubectl get <resource> [options]
```

```
kubectl describe <resource> <name>
```

## Common Resources




## kubectl get Examples

### 1 List all pods

```
$ kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
nginx-6d7c9f8b5-abc12     1/1     Running   0          2m
my-app-7c9d6f4f8-xz7pq    1/1     Running   0          5m
```

### 2 List deployments

```
$ kubectl get deployments
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
my-app   3/3     3            3           10m
nginx    1/1     1            1           15m
```

### 3 List services

```
$ kubectl get services
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP        2d
my-app       NodePort    10.96.123.4    <none>        80:30080/TCP   10m
```

### 4 List all resources in a namespace

```
$ kubectl get all -n my-app
NAME                          READY   STATUS    RESTARTS   AGE
pod/my-app-7c9d6f4f8-xz7pq    1/1     Running   0          5m
deployment.apps/my-app        3/3     Running   0          10m
service/my-app                NodePort  10.96.123.4  80:30080/TCP  10m
```

## kubectl describe Examples

### 1 Describe a pod

```
$ kubectl describe pod my-app-7c9d6f4f8-xz7pq
Name:             my-app-7c9d6f4f8-xz7pq
Namespace:        default
Node:             worker-1/192.168.1.10
Status:           Running
IP:               10.244.1.15
Containers:
  app:
    Image:        nginx:1.25
    State:        Running
    Ready:        True
Events:
  Type    Reason     Age   From               Message
  ----    ------     ---   ----               -------
  Normal  Scheduled  5m    default-scheduler  Successfully assigned
```

### 2 Describe a deployment

```
$ kubectl describe deployment my-app
Name:               my-app
Namespace:          default
Replicas:           3 desired | 3 updated | 3 total | 3 available
StrategyType:       RollingUpdate
MinReadySeconds:    0
Pod Template:
  Containers:
    app:
      Image:        nginx:1.25
      Ports:        80/TCP
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ---   ----                   -------
  Normal  ScalingReplicaSet  5m    deployment-controller  Scaled up
```

## Useful Options

| Option | What It Does |
|---|---|
| -n <namespace> | Specify a namespace |
| -o wide | Show more details (node, IP, etc.) |
| -o yaml | Output in YAML format |
| -o json | Output in JSON format |
| --all-namespaces | Show resources in all namespaces |
| --show-labels | Display resource labels |

## Common Use Cases

- Check if your Pods are running.
- Verify the number of replicas in a Deployment.
- Inspect Service type and exposed ports.
- Investigate why a Pod is not starting.
- Read Kubernetes events for error messages.
- Verify resource configuration and environment variables.
- Understand resource relationships (e.g., which Deployment created a Pod).

## Common Beginner Mistakes

- Using the wrong resource name or namespace.
- Forgetting to check events.
- Not using -n when working in non-default namespaces.
- Only using kubectl get without describe when troubleshooting.
- Misreading the STATUS column.
- Assuming a resource is healthy without checking details.
- Not using -o wide or -o yaml for more information.

VERIQTA

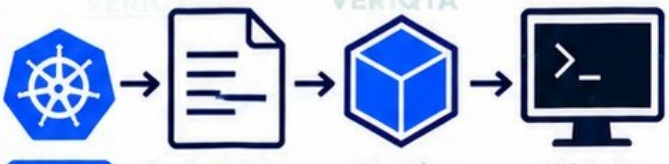
# kubectl logs: Investigating Application Failures

**Understand container logs, previous container logs, and how to investigate application crashes using kubectl logs.**

## What are Container Logs?

- Container logs show the output (stdout and stderr) from your application.
- They help you understand application behavior, errors and crashes.
- You can view current logs, previous logs (from a restarted container), and follow logs in real time.
- Essential for troubleshooting and investigating application failures.

## How kubectl logs Works




## Basic kubectl logs Commands

| Command | What It Does | Example |
|---|---|---|
| kubectl logs <pod> | Show logs for a Pod | kubectl logs my-app-pod |
| kubectl logs <pod> -c <container> | Show logs for a specific container | kubectl logs my-app-pod -c app |
| kubectl logs -f <pod> | Follow logs in real time | kubectl logs -f my-app-pod |
| kubectl logs --tail=100 <pod> | Show last 100 lines | kubectl logs --tail=100 my-app-pod |
| kubectl logs --since=1h <pod> | Show logs from last 1 hour | kubectl logs --since=1h my-app-pod |
| kubectl logs --previous <pod> | Show logs from previous container (after a restart) | kubectl logs --previous my-app-pod |
| kubectl logs <pod> > file.log | Save logs to a file | kubectl logs my-app-pod > app.log |

## Example: Viewing Pod Logs

```
$ kubectl logs my-app-pod

2025-10-02 10:15:23  INFO  Starting application...
2025-10-02 10:15:24  INFO  Connecting to database...
2025-10-02 10:15:25  INFO  Application started
successfully
```

## Follow Logs in Real Time

```
$ kubectl logs -f my-app-pod

2025-10-02 10:16:01  INFO   Processing request
2025-10-02 10:16:02  INFO   Request completed
2025-10-02 10:16:05  ERROR  Failed to connect to DB
2025-10-02 10:16:05  ERROR  Retrying connection...
```

## Previous Container Logs (After a Crash)

```
$ kubectl logs --previous my-app-pod

2025-10-02 09:10:11  INFO   Starting application...
2025-10-02 09:10:12  ERROR  Connection refused
2025-10-02 09:10:12  ERROR  Application crashed
2025-10-02 09:10:12  INFO   Shutting down...
```

## Multiple Containers in a Pod

```
$ kubectl logs my-app-pod -c sidecar

2025-10-02 10:20:01  INFO  Sidecar started
2025-10-02 10:20:02  INFO  Syncing configuration...
2025-10-02 10:20:05  INFO  Configuration updated
```

## Common Use Cases

- Investigate why a Pod is crashing.
- Find application errors and stack traces.
- Check startup and initialization logs.
- View previous logs after a container restart.
- Debug issues in specific containers.
- Monitor logs in real time during testing.
- Save logs for further analysis.

## Common Beginner Mistakes

- Checking the wrong Pod or container.
- Forgetting to use --previous after a crash.
- Not specifying the correct namespace.
- Tailing too many logs (use --tail or --since).
- Assuming logs are persisted (they can be lost after container restart).
- Not checking multiple containers in a Pod.
- Ignoring error messages in the logs.

## Junior Engineer Tip

- Use -f to watch logs while testing.
- Use --previous when a container has restarted.
- Use timestamps in your application logs.
- Combine logs with kubectl describe pod <pod> to get more context.
- If logs are missing, check if the container is running and verify the container name.

VERIQTA

# kubectl exec and debug: Troubleshooting Containers

**Learn the difference between running a command inside a container and using ephemeral debugging containers.**

## What is kubectl exec?

- Runs a command inside a running container.
- Lets you inspect files, run tools, check logs, and troubleshoot issues directly.
- Useful when the container already has the tools you need.
- Does not require a new container.

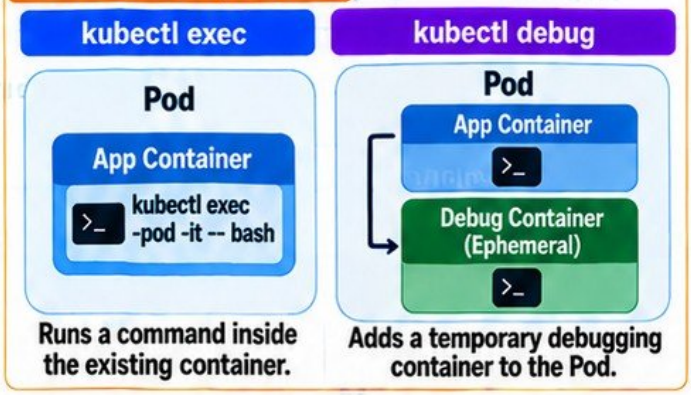
## What is kubectl debug?

- Creates an ephemeral debugging container in the same Pod (or Node).
- Useful when the original container is minimal and does not have debugging tools.
- Lets you troubleshoot without modifying the running container.
- Supports debugging running and crashed Pods.

## kubectl exec vs kubectl debug

| | kubectl exec | kubectl debug |
|---|---|---|
| Purpose | Run a command inside a running container. | Create an ephemeral debugging container. |
| Use case | Container has required tools. | Container is minimal (no debugging tools). |
| Modifies container | No (just runs a command). | No (adds a temporary container). |
| Works on crashed Pod | No (container must be running). | Yes (can debug crashed Pods). |
| Common tools | sh, bash, cat, ls, env, etc. | Full toolbox (e.g, ubuntu, busybox, netshoot). |

## Visual Comparison




Runs a command inside the existing container.

Adds a temporary debugging container to the Pod.

## Common kubectl exec Commands

| Command | What It Does | Exampte |
|---|---|---|
| Execute a shell in a container | Open an interactive shell inside the container. | `kubectl exec -it my-pod --/bin/bash` |
| Run a single command | Run a command without an interactive shell. | `kubectl exec my-pod -- ls /app` |
| Specify container | Exec into a specific container in a Pod. | `kubectl exec -it my-pod -c nginx -- sh` |
| Check environment variables | View environment variables. | `kubectl exec my-pod -- env` |
| View running processes | List processes inside the container. | `kubectl exec my-pod -- ps aux` |
| Check open ports | See listening ports. | `kubectl exec my-pod -- ss -tuln` |

## Common kubectl debug Commands

| Command | What It Does | Example |
|---|---|---|
| Add a debug container to a running Pod | Create an ephemeral container in the Pod. | `kubectl debug my-pod -it --image=ubuntu --target=nginx` |
| Debug a crashed Pod | Add a debug container to a crashed Pod. | `kubectl debug my-pod -it --image=ubuntu` |
| Use a specific tool image | Use a toolbox image (e.g. netshoot). | `kubectl debug my-pod -it --image=nicolaka/netshoot` |
| Debug a node | Create a debug container on a node. | `kubectl debug node/<node-name> -it --image=ubuntu` |
| Run with specific tools | Use an image with troubleshooting tools. | `kubectl debug my-pod -it --image=busybox` |

## Example: Using kubectl exec

```
# Open a shell in the pod
$ kubectl exec -it my-app-pod -- /bin/bash
# Check files
root@my-app-pod:/# ls /app
# Check environment variables
root@my-app-pod:/# env
# Check running processes
root@my-app-pod:/# ps aux
```

## Example: Using kubectl debug

```
# Add an ephemeral debug container
$ kubectl debug my-app-pod -it \
  --image=nicolaka/netshoot --target=nginx

# Inside the debug container
root@my-app-pod:/# curl localhost:8080
root@my-app-pod:/# nslookup kube-dns
root@my-app-pod:/# tcpdump -i any
```

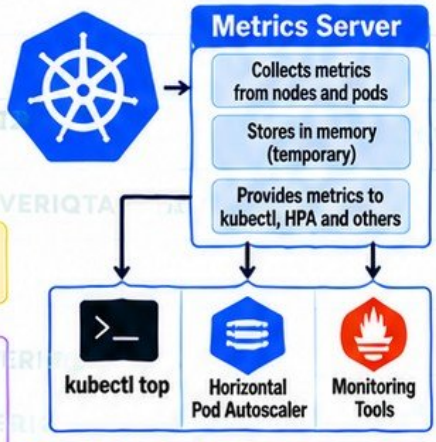
## When to Use What?

| Situation | Use Exec | Use Debug |
|---|---|---|
| Container is running and has necessary tools | ✓ | |
| Container is minimal (no tools) | | ✓ |
| Pod is crashed | | ✓ |
| Need advanced debugging tools | | ✓ |
| Quick inspection (files, env, processes) | ✓ | |

VERIQTA

# kubectl top and Metrics Server

**Understand CPU and memory usage, the role of Metrics Server, and basic resource troubleshooting.**




## What is kubectl top?

- kubectl top shows real-time CPU and memory usage for nodes and pods.
- It gets metrics from Metrics Server.
- Helps you quickly identify high resource usage and potential issues.
- Useful for troubleshooting and capacity planning.
- Requires Metrics Server to be installed in your cluster.

## What is Metrics Server?

- Metrics Server is a lightweight Kubernetes add-on.
- Collects CPU and memory metrics from nodes and pods.
- Provides metrics to kubectl top, Horizontal Pod Autoscaler (HPA) and other tools.
- Stores metrics in memory only (short duration).
- Not a long-term monitoring solution.
- Easy to install and works in most Kubernetes clusters.

## Basic kubectl top Commands

| Command | What It Shows | Example |
|---|---|---|
| kubectl top nodes | CPU and memory usage for nodes | kubectl top nodes |
| kubectl top pods | CPU and memory usage for all pods | kubectl top pods |
| kubectl top pod <pod-name> | Usage for a specific pod | kubectl top pod my-app-pod |
| kubectl top pod -n <namespace> | Usage in a specific namespace | kubectl top pod -n production |
| kubectl top --containers | Show container level usage | kubectl top pod my-app-pod --containers |

## Install Metrics Server (if not installed)

```
# Install Metrics Server
kubectl apply -f \\
  https://github.com/kubernetes-sigs/metrics-server/
  releases/latest/download/components.yaml

# Check if it is running
kubectl get pods -n kube-system | grep metrics-server
```

## Verify Metrics Server

```
# Check deployment
kubectl get deploy -n kube-system \
metrics-server
# Check pods
kubectl get pods -n kube-system \
  -l k8s-app=metrics-server
# Check logs (if needed)
kubectl logs -n kube-system \
  deploy/metrics-server
```

## Example: kubectl top nodes

```
$ kubectl top nodes
NAME                 CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
kind-control-plane   250m         12%    1024Mi          45%
kind-worker          180m         9%     768Mi           32%
```

## Example: kubectl top pods

```
$ kubectl top pods
NAME                          CPU(cores)   MEMORY(bytes)
my-app-6d7c9f8b5-abc12        50m          128Mi
nginx-5f8d9c7d6-xyz34         20m          64Mi
redis-77c8d9f6f-rtg56         10m          32Mi
```

## Example: Container level metrics

```
$ kubectl top pod my-app-6d7c9f8b5-abc12 --containers
NAME        CPU(cores)   MEMORY(bytes)
my-app      45m          120Mi
sidecar     5m           20Mi
```

## Troubleshooting Tips

- If kubectl top returns an error, check if Metrics Server is installed and running.
- Ensure your cluster version is supported.
- Check Metrics Server logs for issues.
- High CPU or memory usage may indicate:
  - Resource limits too low
  - Memory leaks
  - Too many requests
  - Inefficient application code

## Common Errors and Fixes

| Error Message | Fix |
|---|---|
| error: metrics not available yet | Wait a few minutes for Metrics Server to collect metrics. |
| error: the server doesn't have a resource type "pods" | Install Metrics Server in your cluster. |
| No resources found | Check if you are in the correct namespace. |
| Metrics show 0 or <unknown> | Verify Metrics Server is running and has proper permissions. |

## When to Use kubectl top?

- Quickly check resource usage.
- Identify high resource consumers.
- Troubleshoot performance issues.
- Validate resource requests and limits.
- Useful during incident investigation.

## Limitations

- Metrics Server stores metrics in memory only (short duration).
- Not suitable for long-term monitoring.
- Limited historical data.
- For advanced monitoring, use tools like Prometheus and Grafana.

## Junior Engineer Tip

- Always have Metrics Server installed in your cluster.
- Use kubectl top when debugging high CPU or memory issues.
- Combine with kubectl describe and kubectl logs to investigate further.
- Use proper resource requests and limits for production workloads.

VERIQTA

# K9s: Terminal-Based Kubernetes Management

**Learn how K9s helps engineers navigate workloads, inspect resources, view logs, and investigate cluster problems.**

## What is K9s?

- K9s is a terminal-based UI for managing Kubernetes clusters.
- Provides an interactive and easy way to navigate workloads, inspect resources, view logs and troubleshoot issues.
- Popular with DevOps, SREs and developers for daily cluster operations.
- Fast, lightweight and works on Linux, macOS and Windows.
- Does not replace kubectl, it complements it.

## Key Features

- Navigate all Kubernetes resources.
- View resource status, details and logs.
- Execute commands inside containers.
- Port forward and access services.
- View events and troubleshoot issues.
- Multi-namespace support.
- Real-time cluster view with filtering and search.
- Keyboard-driven and highly productive.

## K9s Interface Overview




## Install K9s

### Linux (using curl)

```
curl -sS https://webinstall.dev/k9s | bash
```

### macOS (using Homebrew)

```
brew install derailed/k9s/k9s
```

### Windows (using Scoop)

```
scoop install k9s
```

### Verify Installation

```
k9s version
```

## K9s Main Screen (Pods View)

```
Context:  kind-kind          <0> all namespaces     <a> Attach
Cluster:  kind-kind          <1> default            <d> Describe
User:     kind-kind          <2> kube-system        <e> Edit
K9s Rev:  v0.32.5            <3> kube-public        <l> Logs
K8s Rev:  v1.28.0            <4> kube-node-lease    <s> Shell
CPU:      12%                                       <f> Port-Forward
MEM:      28%                                       <?> Help
```

Pods(all namespaces)

| NAMESPACE | NAME | READY | STATUS | RESTARTS | AGE |
|---|---|---|---|---|---|
| default | my-app-6d7c9f8b5-abc12 | 1/1 | Running | 0 | 2m |
| default | my-app-6d7c9f8b5-def34 | 1/1 | Running | 0 | 2m |
| kube-system | coredns-5d78c9869-xyz12 | 1/1 | Running | 0 | 1d |
| kube-system | metrics-server-6fdf8 | 1/1 | Running | 0 | 1d |
| kube-system | kube-proxy-abcde | 1/1 | Running | 0 | 1d |

## Useful K9s Shortcuts

| Key | Action |
|---|---|
| : | Open command menu |
| / | Search resources |
| ? | Show help |
| d | Describe resource |
| e | Edit resource (YAML) |
| l | View logs |
| s | Open shell in container |
| f | Port forward |
| x | Delete resource |
| 0 | All namespaces |
| 1..9 | Switch namespace |
| Ctrl + c | Quit K9s |

## Viewing Logs in K9s

Logs(my-app-6d7c9f8b5-abc12)

```
2025-10-02 10:15:23  INFO   Starting application...
2025-10-02 10:15:24  INFO   Connecting to database...
2025-10-02 10:15:25  INFO   Application started
2025-10-02 10:16:01  INFO   Processing request
2025-10-02 10:16:05  ERROR  Failed to connect to DB
2025-10-02 10:16:05  ERROR  Retrying connection...
```

## Shell into a Container

Shell(my-app-6d7c9f8b5-abc12)

```
/ # ls
app    bin    etc    lib    tmpr
/ # ps aux
  PID   USER     TIME   COMMAND
    1   root     0:00   /app
    7   root     0:00   nginx
```

## Common Use Cases

- Quickly view the status of all workloads.
- Investigate failing pods using logs and shell access.
- Check events and resource details.
- Port forward a service for local testing.
- Switch between namespaces and contexts.
- Delete or edit resources when needed.
- Monitor cluster health during incidents.

## Benefits

- Faster navigation than using only kubectl.
- Easy to inspect and troubleshoot resources.
- View logs, events and resource details in one place.
- Improves productivity during incident response.
- Lightweight and simple to use.

## Tips for Junior Engineers

- Start with read-only actions (view, describe, logs).
- Use K9s to learn the Kubernetes resources and structure.
- Check logs and events when something is failing.
- Use port-forward to test applications locally.
- Combine K9s with kubectl for advanced tasks.

VERIQTA

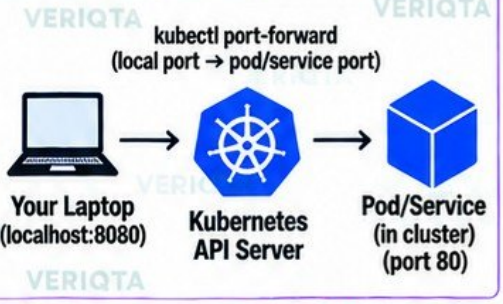
# kubectl port-forward: Accessing Applications

**Understand how port forwarding works, when engineers use it, and why it should not replace production ingress.**

## What is kubectl port-forward?

- Forwards a local port on your machine to a port on a Pod or Service in the cluster.
- Allows you to access an application without exposing it publicly.
- Useful for development, testing, troubleshooting and quick access.
- Works over the Kubernetes API server (secure tunnel).
- Not a replacement for production ingress (Ingress, LoadBalancer or Service).

## How It Works




## Common Use Cases

- Access an internal web UI (e.g. Kubernetes dashboard).
- Test an application locally.
- Debug an internal service without exposing it.
- Access a database temporarily.
- Troubleshoot connectivity issues.
- Inspect an application running in a private namespace.

## Port Forward to a Pod

```
$ kubectl port-forward my-app-pod 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 127.0.0.1:52344
Handling connection for 127.0.0.1:52345
```

Now open in your browser:
http://localhost:8080

## Port Forward to a Service

```
$ kubectl port-forward svc/my-app 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Access the service at:
http://localhost:8080

## Port Forward a Different Local Port

```
$ kubectl port-forward svc/my-app 3000:80
Forwarding from 127.0.0.1:3000 -> 80
Forwarding from [::1]:3000 -> 80
```

Access at:
http://localhost:3000

## Forward to a Database (Example)

```
$ kubectl port-forward svc/postgres 5432:5432
Forwarding from 127.0.0.1:5432 -> 5432
Forwarding from [::1]:5432 -> 5432
```

Now connect using your local client:
host: localhost
port: 5432

## Forward to a Specific Namespace

```
$ kubectl port-forward -n monitoring \
  svc/grafana 3000:80
Forwarding from 127.0.0.1:3000 -> 80
Forwarding from [::1]:3000 -> 80
```

Access at:
http://localhost:3000

## Forward Multiple Ports

```
$ kubectl port-forward svc/my-app \
  8080:80 8443:443
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from 127.0.0.1:8443 -> 443
```

Access at:
http://localhost:8080 (HTTP)
https://localhost:8443 (HTTPS)

## Common kubectl port-forward Options

| Option | What It Does | Example |
|---|---|---|
| -n <namespace> | Specify a namespace | kubectl port-forward -n dev .. |
| --address 0.0.0.0 | Listen on all interfaces (not recommended) | kubectl port-forward --address 0.0.0.0 .. |
| --pod-running-timeout | Wait for pod to be running | kubectl port-forward --pod-running-timeout=60s .. |
| --disconnect-timeout | Idle timeout for connections | kubectl port-forward --disconnect-timeout =10m .. |

## Why Not Use It in Production?

- Not designed for external users.
- Requires an active kubectl session (not always available).
- Not highly available or scalable.
- Traffic goes through your local machine and the API server.
- Not a replacement for Ingress, LoadBalancer or a proper Service.

## Troubleshooting Tips

- Make sure the Pod/Service is running.
- Check the correct namespace.
- Verify the local port is not already in use.
- Confirm you have access to the cluster.
- Check API server connectivity if it faiils.

## Common Errors

| Error Message | Fix |
|---|---|
| error: pod not found | Check the pod name and namespace. |
| error: unable to upgrade connection | Check cluster connectivity and permissions. |
| bind: address already in use | Use a different local port. |
| connection refused | Verify the pod/service is running and the correct port. |

## Junior Engineer Tips

- Use port-forward for temporary access.
- Always specify the namespace.
- Use it for debugging and testing only.
- Prefer Ingress or LoadBalancer for production access.
- Combine with kubectl logs, exec and describe for troubleshooting.

VERIQTA

# kubectl auth can-i: Checking Permissions

**Learn how to inspect Kubernetes Role-Based Access Control (RBAC) permissions and troubleshoot authorization failures.**

## What is kubectl auth can-i?

- Checks if your current user or service account is allowed to perform a specific action.
- Uses the Kubernetes RBAC rules (Roles, ClusterRoles, RoleBindings, ClusterRoleBindings).
- Helps you troubleshoot "Forbidden" errors.
- Useful for developers, SREs and cluster administrators.
- Does not actually perform the action, it only checks permission.

## How It Works




## Common Use Cases

- Verify what you can and cannot do.
- Troubleshoot "Forbidden" errors.
- Check permissions in a specific namespace.
- Validate permissions for another user or service account.
- Inspect cluster-wide permissions.
- Helpful during RBAC troubleshooting and access reviews.

## Basic Syntax

```
$ kubectl auth can-i [verb] [resource] [flags]
```

## Example Output

```
$ kubectl auth can-i get pods
yes
```

```
$ kubectl auth can-i delete pods
no
```

## Common Examples

| Command | What It Checks |
|---|---|
| kubectl auth can-i get pods | Can you get Pods in current namespace? |
| kubectl auth can-i list deployments | Can you list Deployments? |
| kubectl auth can-i create pods | Can you create Pods? |
| kubectl auth can-i delete svc | Can you delete Services? |
| kubectl auth can-i get secrets | Can you get Secrets? |
| kubectl auth can-i --all-namespaces get pods | Can you get Pods in all namespaces? |

## Check in a Specific Namespace

```
$ kubectl auth can-i get pods -n dev
$ kubectl auth can-i create deployments -n production
```

## Check as Another User

```
$ kubectl auth can-i get pods \
 --as=john
$ kubectl auth can-i delete pods \
 --as=dev-user -n dev
```

## Check Resource Types and API Groups

```
$ kubectl auth can-i get pods --api-group=""
$ kubectl auth can-i get deployments --api-group=apps
$ kubectl auth can-i get ingress --api-group=networking.k8s.io
```

## Real Examples

```
$ kubectl auth can-i get configmaps -n dev
$ kubectl auth can-i patch deployments -n prod
$ kubectl auth can-i list nodes
$ kubectl auth can-i create persistentvolumeclaims
$ kubectl auth can-i get events --all-namespaces
```

## Common Errors and Meaning

| Error Message | Meaning |
|---|---|
| no | You do not have permission to perform this action. |
| Forbidden | RBAC is blocking the request. |
| error: the server doesn't have a resource type | Incorrect resource name or API group. |
| error: unknown flag | Invalid flag or syntax. |

## Troubleshooting Tips

- Confirm the correct resource name (e.g. pods, deployments, services).
- Check the namespace.
- Verify the API group if needed.
- Use --as to test another user or service account.
- Review Roles, ClusterRoles and Bindings.
- Check for typos in resource names.

## Junior Engineer Tip

- If you get "no", check with your cluster administrator.
- Use kubectl auth can-i to quickly understand what you can do.
- Combine this with kubectl get, describe and logs when troubleshooting access issues.
- Always specify the namespace when working in multi-tenant clusters.
- Understand that RBAC controls what resources you can access, not whether they exist.

VERIQTA

# kubectl diff and Rollout Commands

**Understand how to preview configuration changes, monitor deployments, inspect rollout history, and undo supported Deployment revisions.**

## What is kubectl diff?

- Compares your local manifest with the current configuration in the cluster.
- Shows what changes will be applied before you run kubectl apply.
- Helps you catch mistakes early.
- Useful for safer and more controlled deployments.
- Requires server-side diff support (works on most modern clusters).

## What are Rollout Commands?

- Manage and monitor the rollout process of Deployments.
- Check rollout status in real time.
- View rollout history and revision details.
- Undo to a previous revision (if supported).
- Useful for troubleshooting failed deployments and quickly recovering to a stable version.

## Common Use Cases

- Preview changes before applying.
- Verify what will be updated in the cluster.
- Monitor deployment progress.
- Check rollout history and revision details.
- Rollback to a previous version when an issue occurs.
- Troubleshoot stuck or failed rollouts.
- Use during production releases for safer deployments.

## kubectl diff Example

```
$ kubectl diff -f deployment.yaml
diff -u -N /tmp/LIVE /tmp/MERGED
--- /tmp/LIVE
+++ /tmp/MERCED
@@ -8,7 +8,7 @@
  spec:
-   replicas: 2
+   replicas: 3
    template:
```

Shows the difference between your local file and the current cluster configuration.

## Common Rollout Commands

| Command | What It Does |
|---|---|
| kubectl rollout status deployment/<name> | Watch the rollout progress in real time. |
| kubectl rollout history deployment/<name> | Show revision history of the Deployment. |
| kubectl rollout history deployment/<name> --revision=2 | Show details of a specific revision. |
| kubectl rollout undo deployment/<name> | Rollback to the previous revision. |
| kubectl rollout undo deployment/<name> --to-revision=2 | Rollback to a specific revision. |
| kubectl rollout restart deployment/<name> | Restart the Deployment (pods are recreated). |

## Check Rollout Status

```
$ kubectl rollout status
  deployment/my-app
deployment "my-app" successfully
rolled out
```

## View Rollout History

```
$ kubectl rollout history
  deployment/my-app
REVISION     CHANGE-CAUSE
1            <none>
2            Update image to v2
3            Update replicas to 3
```

## View Specific Revision Details

```
$ kubectl rollout history
  deployment/my-app --revision=2
deployment.apps/my-app
REVISION: 2
CHANGE-CAUSE:
  Update image to v2
Pod Template:
  Containers:
    my-app:
      Image:  nginx:1.21
```

## Undo a Rollout (Rollback)

```
$ kubectl rollout undo
  deployment/my-app
deployment.apps/my-app
rolled back

$ kubectl rollout undo
  deployment/my-app --to-revision=2
deployment.apps/my-app
rolled back to revision 2
```

## Restart a Deployment

```
$ kubectl rollout restart
  deployment/my-app
deployment.apps/my-app restarted
```

## Important Notes

- kubectl diff does not make any changes.
- Rollout history is only availalde for Deployments (and some other resources).
- Rollback only works if the previous ReplicaSet still exists.
- Use meaningful change-cause annotations (e.g. --record or annotation) for better trackirry.

## Common Errors and Fixes

| Error Message | Fix |
|---|---|
| error: unknown command "diff" | Install a newer kubectl version (1.18+). |
| The server could not find the requested resource (rollout) | Ensure you are using a Deployment and correct API version. |
| error: revision not found | Check available revisions using kubectl rollout history. |

## Tips for Junior Engineers

- Always run kubectl diff before applying changes.
- Use kubectl rollout status to confirm deployments are healthy.
- Check rollout history to understand past changes.
- Use rollback if a new version causes issues.
- Add clear change messages for better tracking.
- Combine with kubectl get pods and kubectl describe for troubleshooting.

## Real-World Workflow

1. Make changes to your manifest.
2. Run kubectl diff -f <file>.
3. Apply changes: kubectl apply -f <file>.
4. Monitor rollout: kubectl rollout status.
5. If issues occur, check history and rollback if needed.
6. Verify the application is working.

VERIQTA

# kubectx and kubens: Switching Contexts and Namespaces

**Learn how these tools simplify switching between Kubernetes contexts and namespaces while avoiding changes to the wrong environment.**

## What are kubectx and kubens?

- **kubectx:** A small command-line tool for quickly switching between Kubernetes contexts.
- **kubens:** A tool for quickly switching between namespaces.
- They make it easier and faster to navigate multiple clusters and namespaces.
- Help prevent accidental changes to the wrong environment.
- Work with your existing kubeconfig file, they do not replace kubectl.

## Key Benefits

- Quick and simple to use.
- List and switch contexts and namespaces with short commands.
- Reduces the risk of working in the wrong cluster or namespace.
- Improves productivity for DevOps engineers and SREs.
- Works on Linux, macOS and Windows.
- Popular tools in real-world Kubernetes environments.

## Install kubectx and kubens

### On Linux (using kubectx/kubens)

```
# Install using git
git clone https://github.com/ahmetb/kubectx
sudo ln -s kubectx/kubectx /usr/local/bin/
sudo ln -s kubectx/kubens /usr/local/bin/
```

### On macOS (using Homebrew)

```
brew install kubectx
```

### On Windows (using Scoop)

```
scoop install kubectx
```

## kubectx Commands (Contexts)

```
$ kubectx          # List all contexts
$ kubectx dev      # Switch to a context
$ kubectx prod     # Switch to another context
$ kubectx -        # Switch to previous context
$ kubectx -c       # Show current context
$ kubectx <name>   # Switch to specific context
```

## Example: Switching Contexts

```
$ kubectx
    dev-cluster
    staging-cluster
  * prod-cluster     (current)
$ kubectx dev
Switched to context "dev-cluster".
```

## kubens Commands (Namespaces)

```
$ kubens           # List namespaces
$ kubens dev       # Switch to a namespace
$ kubens kube-system  # Switch to another
$ kubens -         # Switch to previous
$ kubens -c        # Show current namespace
$ kubens <name>    # Switch to specific
```

## Example: Switching Namespaces

```
$ kubens
    default
    kube-system
    kube-public
  * dev          (current)
    prod
$ kubens prod
Switched to namespace "prod".
```

## Check Current Context and Namespace

```
$ kubectx -c
dev-cluster

$ kubens -c
prod
```

## Visual Prompt in Your Shell

```
# Many engineers customize their shell
# prompt to show context and namespace
(dev-cluster:prod) $ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
my-app-7d5c6  1/1     Running   0          2m
```

## Real-World Use Case




1. Use kubectx to switch to the correct cluster (e.g. dev, staging, prod).
2. Use kubens to switch to the correct namespace.
3. Verify the current context and namespace before making changes.
4. Run kubectl commands safely.

## Common Mistakes

- Working in the wrong context or namespace.
- Forgetting to verify the current context.
- Making changes in production by mistake.
- Assuming the default namespace is correct.
- Not using these tools and relying only on kubectl config commands.

## Related kubectl Commands

```
$ kubectl config get-contexts  # List contexts
$ kubectl config use-context <name>
$ kubectl config current-context
$ kubectl get namespaces
$ kubectl config set-context --current
  --namespace=<name>
```

## Tips for Junior Engineers

- Always check the current context and namespace before running any command.
- Use kubectx and kubens to switch quickly.
- Set up shell prompts to show your context and namespace.
- Double-check when working in production.
- Develop a habit of verifying your environment.

## Key Takeaways

- kubectx and kubens make it easy to switch between contexts and namespaces.
- They help prevent accidental changes.
- Always verify your current context and namespace.
- Use them as part of your daily workflow.
- They are simple but powerful tools for multi-cluster and multi-namespace environments.